



МИНОБРНАУКИ РОССИИ

**Федеральное государственное бюджетное образовательное учреждение
высшего образования**

«МИРЭА – Российский технологический университет»

РТУ МИРЭА

Институт Информационных технологий

**Кафедра Математического обеспечения и стандартизации информационных
технологий**

Отчет по практическим работам №9-10

по дисциплине

«Системная и программная инженерия»

Выполнили:

Студенты группы ИКБО-66-23

**Гордильо А.
Смирнов А.Ю.
Ковалёв А.Э.
Тименко И.А.**

Проверил:

Мельников Д.А.

Москва 2026

Оглавление

1 Выбор методологии управления процессом разработки.....	3
2 Создание удалённого Git-репозитория.....	4
3 Выбранные инструменты разработки программного обеспечения.....	4
4 Документация разработчика.....	5
5 Документация пользователя.....	7
6 ЗАКЛЮЧЕНИЕ.....	9

1 Выбор методологии управления процессом разработки

Выбранная методология: Agile + итерационная модель.

Для разработки программного продукта для планирования встреч сотрудников была выбрана комбинированная методология Agile с использованием итерационного подхода. Данный выбор обусловлен тем, что система состоит из двух взаимосвязанных частей: мобильного Android-приложения и серверной части. В процессе разработки необходимо постепенно реализовывать и проверять отдельные функции: регистрацию и авторизацию, управление профилем, создание встреч, работу с приглашениями и просмотр расписания.

Таблица 1 – причина выбора модели

Модель	Почему не подходит / подходит
Waterfall	Не подходит, так как требования к интерфейсу, API и логике взаимодействия клиента с сервером могут уточняться в процессе разработки.
Scrum	Частично подходит, так как позволяет работать спринтами, но для учебного проекта может быть избыточен из-за необходимости строгого распределения ролей и регламентов.
Kanban	Частично подходит для отслеживания задач, но в чистом виде слишком свободен и может привести к потере контроля над этапами разработки.
Итерационная	Подходит, так как каждая итерация позволяет получить рабочую часть системы: авторизацию, профиль, встречи, приглашения, расписание.
Спиральная	Частично подходит, так как позволяет учитывать риски, например ошибки авторизации, конфликты расписания и проблемы интеграции клиента с сервером.
Agile	Подходит, так как предполагает гибкую разработку, постоянную проверку результата и взаимодействие участников команды.

Модель	Почему не подходит / подходит
Lean	Не является основной моделью, так как проект требует не только минимизации лишних действий, но и полноценной реализации клиентской и серверной логики.

Разработка приложения выполнялась поэтапно. На первых этапах были определены основные требования к системе и пользовательские сценарии. Далее была выполнена декомпозиция задач на клиентскую и серверную части. После этого команда приступила к реализации отдельных функций приложения.

Основные итерации разработки:

1. Реализация регистрации и авторизации пользователей.
2. Реализация личного профиля сотрудника.
3. Реализация создания встречи с выбором даты, времени и участников.
4. Реализация просмотра активных приглашений.
5. Реализация ответа на приглашение: принятие или отклонение.
6. Реализация просмотра расписания встреч по дням, неделям и месяцам.
7. Интеграция мобильного приложения с серверным API.
8. Подготовка документации разработчика и пользователя.

Один из принципов Agile заключается во взаимодействии участников команды между собой. В рамках проекта это особенно важно, так как frontend-часть зависит от API backend-части. Такой подход позволяет быстрее находить ошибки, согласовывать формат данных и постепенно дорабатывать функциональность приложения.

2 Создание удалённого Git-репозитория

Выбранный сервис: GitHub.

Для хранения исходного кода проекта были использованы два удалённых репозитория: один для серверной части приложения, второй для клиентской Android-части.

№	Назначение репозитория	Название репозитория	Ссылка	Статус
1	Серверная часть	Android-Bootcamp-2026-Backend	https://github.com/dekoart/Android-Bootcamp-2026-Backend/tree/team-backend-boys	Публичный
2	Клиентская Android-часть	Android-Bootcamp-2026-Frontend	https://github.com/dekoart/Android-Bootcamp-2026-Frontend/tree/team-backend-boys	Публичный

Рис. 1 – Репозиторий GitHub BackEnd

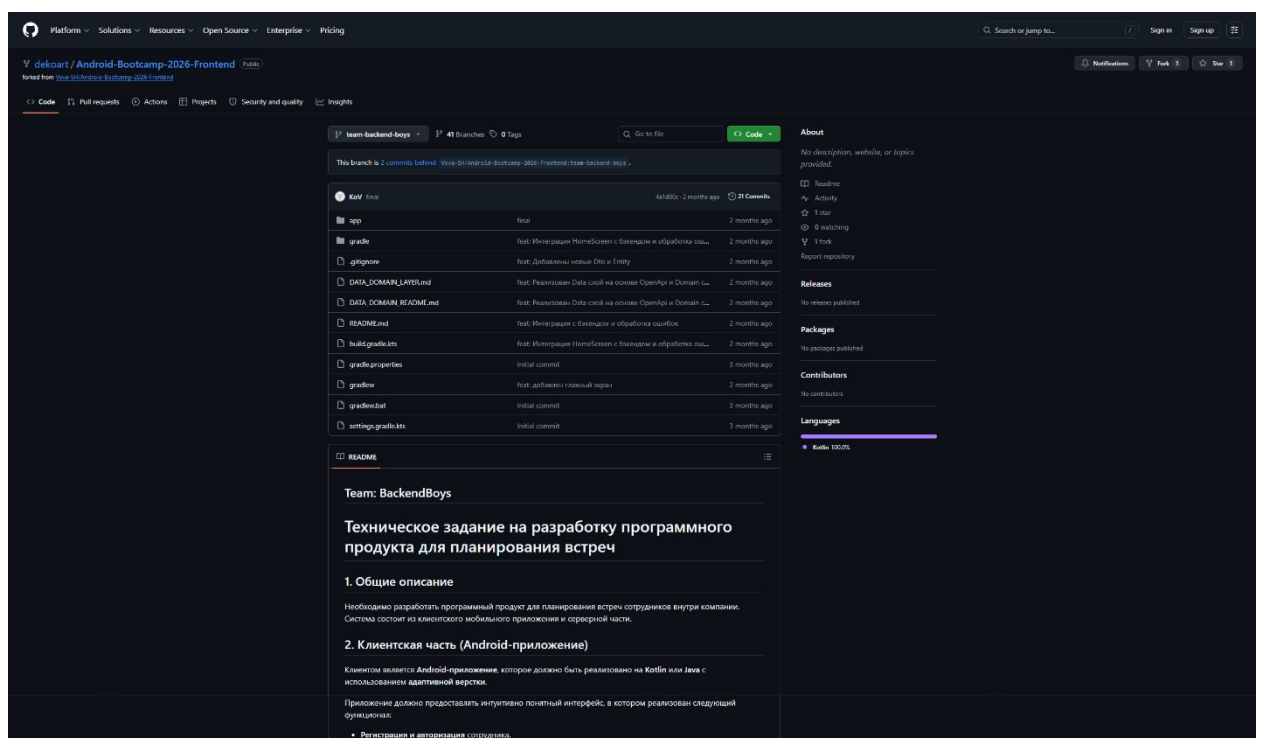


Рис. 2 – Репозиторий GitHub FrontEnd

3 Выбранные инструменты разработки программного обеспечения

Для разработки программного продукта были выбраны инструменты, соответствующие архитектуре проекта. Система состоит из мобильного

Android-приложения и серверной части, поэтому набор инструментов разделён на frontend, backend, базу данных, документацию и контроль версий.

Таблица 3 – Инструменты разработки

№	Категория	Инструмент	Обоснование
1	Клиентская часть	Android Studio	Основная среда разработки Android-приложений, поддерживает Kotlin, Gradle, эмулятор и инструменты отладки.
2	Язык клиентской части	Kotlin	Современный язык для Android-разработки, поддерживает лаконичный синтаксис и хорошо интегрируется с Android SDK.
3	UI-фреймворк	Jetpack Compose	Позволяет создавать современный декларативный интерфейс мобильного приложения.
4	UI-компоненты	Material 3	Используется для создания удобного и современного пользовательского интерфейса.
5	Архитектура frontend	Clean Architecture	Позволяет разделить приложение на слои Data, Domain и Presentation, что упрощает сопровождение и тестирование.
6	Работа с сетью	Retrofit + OkHttp	Используется для взаимодействия Android-приложения с серверным REST API.
7	Сериализация данных	Kotlinx Serialization	Позволяет преобразовывать JSON-ответы сервера в Kotlin-модели и обратно.
8	Dependency Injection	Hilt	Упрощает управление зависимостями в Android-приложении.
9	Локальное хранение	DataStore Preferences	Используется для хранения access-токена, refresh-токена и данных авторизации.
10	Серверная часть	Spring Boot	Позволяет быстро создавать серверные приложения и REST API.
11	Язык серверной части	Java	Используется для реализации backend-логики приложения.
12	Работа с базой данных	Spring Data JPA	Упрощает выполнение операций с базой данных через репозитории и сущности.
13	База данных	PostgreSQL	PostgreSQL используется как основная СУБД, может применяться для локальной разработки и тестирования.
14	Миграции базы данных	Liquibase	Позволяет управлять структурой базы данных и изменениями схемы.
15	Безопасность	Spring Security + JWT	Используется для авторизации пользователей и защиты API.
16	Документация API	Springdoc OpenAPI / Swagger UI	Позволяет автоматически формировать документацию REST API.
17	Сборка backend	Maven	Используется для управления зависимостями и сборки серверного приложения.
18	Сборка frontend	Gradle	Используется для сборки Android-

			приложения и подключения зависимостей.
19	Контейнеризация	Docker	Позволяет запускать backend-приложение в контейнере. Docker compose
20	Контроль версий	Git + GitHub	Используется для хранения исходного кода, командной работы и истории изменений.
21	Проектирование UI/UX	Figma	Используется для проектирования интерфейса мобильного приложения.
22	Проектирование БД	dbdiagram.io	Используется для визуального проектирования структуры базы данных.

Выбранные инструменты позволяют реализовать полный цикл разработки приложения: от проектирования интерфейса и базы данных до реализации серверной логики, мобильного клиента, API и документации.

4 Документация разработчика

Документация разработчика предназначена для описания структуры, архитектуры и логики работы программного продукта для планирования встреч сотрудников.

В рамках выполнения практической работы документация разработчика была представлена в нескольких форматах:

1. README-документация в GitHub-репозиториях.
2. Документация по архитектуре клиентской части.
3. Документация REST API через Swagger UI / OpenAPI.
4. Описание структуры базы данных.
5. Описание основных пользовательских сценариев и use case.

Серверная часть приложения реализована на Spring Boot и предоставляет REST API, необходимый для работы мобильного приложения. API используется для регистрации и авторизации пользователей, получения и редактирования профиля, создания встреч, просмотра приглашений, ответа на приглашения и просмотра расписания.

Клиентская часть Android-приложения построена с использованием Clean Architecture. В приложении выделены следующие слои:

- **Domain** – бизнес-логика, модели данных и use case.
- **Data** – работа с удалённым API, DTO-моделями, мапперами и локальным хранилищем.
- **Presentation** – пользовательский интерфейс приложения.

В документации frontend-части описаны основные модели данных:

- **AuthTokens** – модель токенов авторизации;
- **User** – модель пользователя;
- **Meeting** – модель встречи;
- **Invitation** – модель приглашения;
- **Participant** – модель участника встречи.

Также описаны основные репозитории:

- **AuthRepository** – регистрация, вход, выход, обновление токенов;
- **ProfileRepository** – получение и редактирование профиля;
- **MeetingRepository** – создание и получение встреч;
- **InvitationRepository** – получение приглашений и ответ на них.

REST API серверной части включает следующие группы endpoint-ов:

- **AuthApi** – регистрация, вход, обновление токенов, выход;
- **ProfileApi** – получение и обновление профиля;
- **MeetingApi** – создание встречи, получение встреч, отмена и удаление встречи, получение свободного времени;
- **InvitationApi** – получение приглашений и ответ на приглашение.

В результате подготовки документации разработчика были получены:

- описание архитектуры приложения;
- описание клиентской и серверной частей;

- описание моделей данных;
- описание API;
- описание структуры базы данных;
- описание пользовательских сценариев;
- навигация по исходному коду проекта.

3. Серверная часть

Сервер представляет собой **микросервис на Spring Boot**, работающий с СУБД **PostgreSQL** или **H2**.

Серверное приложение должно:

- Предоставлять API, необходимый для полноценной работы мобильного приложения.
- Осуществлять все основные операции с данными в базе через **Spring Data JPA**.
- Использовать **Liquibase** для создания схемы базы данных и ее предзаполнения.

Рисунок 3 – README-документация серверной части проекта.

Data и Domain слои - Документация

Обзор архитектуры

Приложение следует Clean Architecture с разделением на слои:

- **Domain** - бизнес-логика и модели данных
- **Data** - работа с данными (API, DataStore)
- **Presentation** - UI (будет реализован отдельно)

Domain слой

Модели данных (domain/model/)

AuthTokens

- `userId: UUID` - ID пользователя
- `username: String` - Имя пользователя
- `email: String` - Email
- `accessToken: String` - Access token для API
- `refreshToken: String` - Refresh token
- `accessTokenExpiresAt: LocalDateTime` - Время истечения access токена
- `refreshTokenExpiresAt: LocalDateTime` - Время истечения refresh токена

User

- `id: UUID` - ID пользователя
- `username: String` - Имя пользователя
- `email: String` - Email
- `avatarUrl: String?` - URL аватара (опционально)

Meeting

- `id: UUID` - ID встречи
- `organizerId: UUID` - ID организатора
- `organizerUsername: String` - Имя организатора
- `title: String` - Название встречи
- `description: String?` - Описание (опционально)
- `location: String?` - Место проведения (опционально)
- `startTime: LocalDateTime` - Время начала (должно быть ровно в начале часа)
- `endTime: LocalDateTime` - Время окончания
- `status: MeetingStatus` - Статус (SCHEDULED, CANCELLED, COMPLETED)
- `createdAt: LocalDateTime` - Время создания
- `updatedAt: LocalDateTime` - Время обновления
- `participants: List<Participant>` - Список участников

Рисунок 4 – Документация Data и Domain слоёв клиентской части.

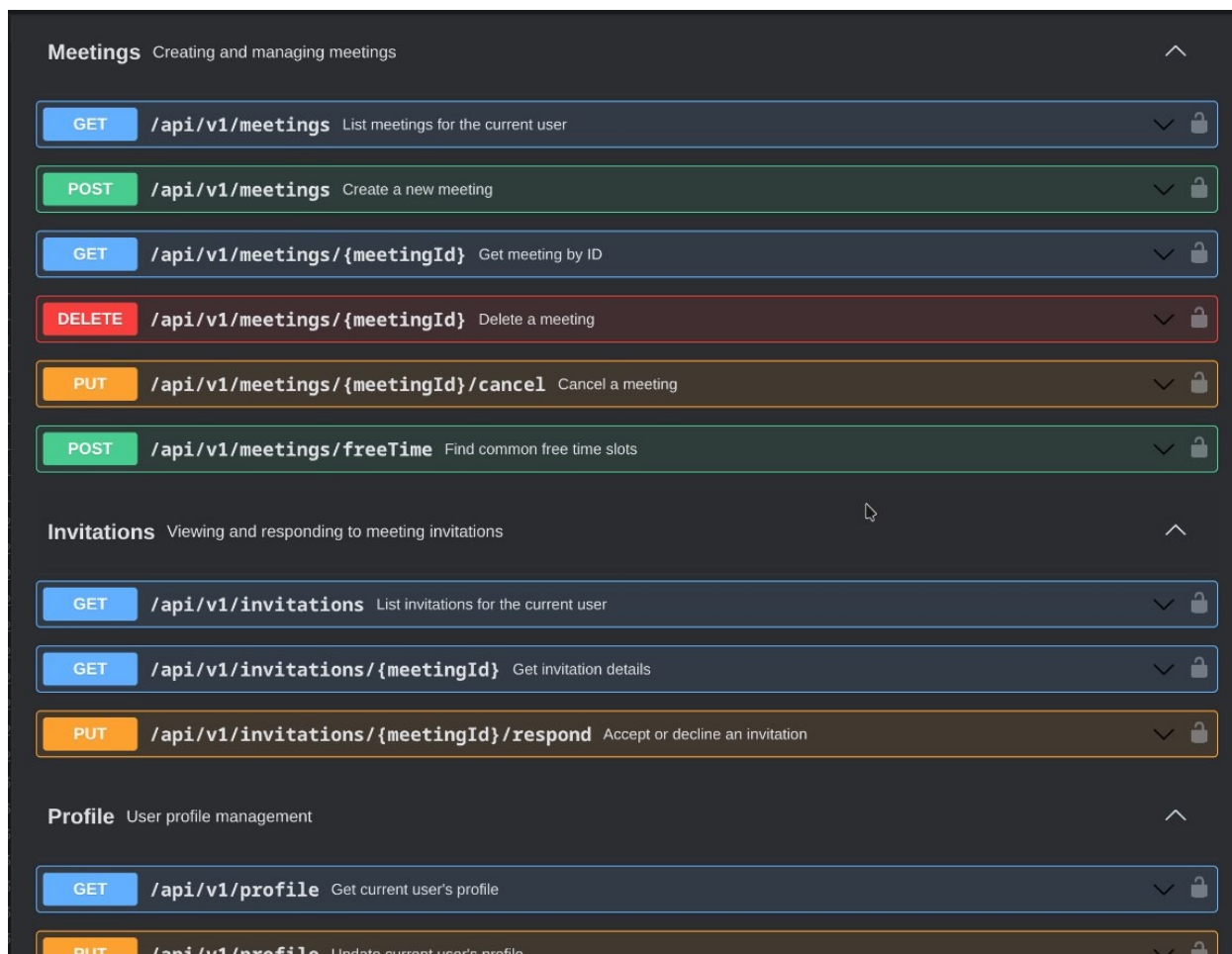


Рисунок 5 – Swagger UI с описанием REST API серверной части.

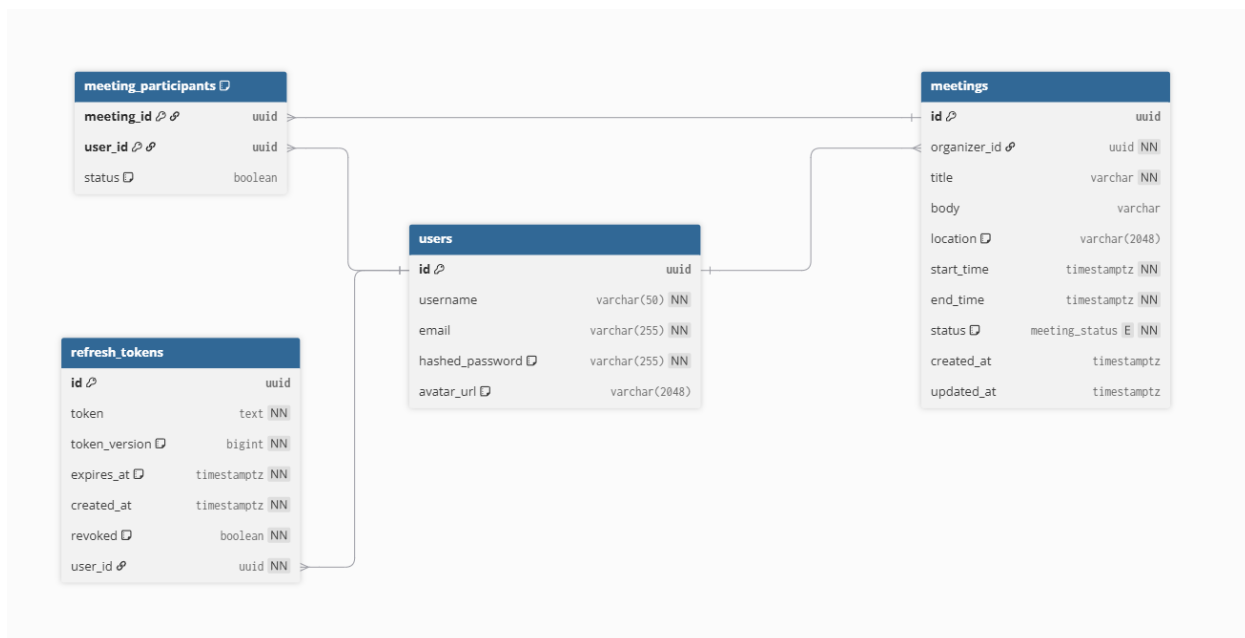


Рисунок 6 – Структура базы данных приложения.

5 Документация пользователя

Документация пользователя предназначена для объяснения принципов работы приложения и описания основных действий, которые пользователь может выполнить в системе.

Программный продукт предназначен для планирования встреч сотрудников внутри компании. Пользователь может зарегистрироваться, авторизоваться, настроить личный профиль, создать встречу, пригласить других сотрудников, просмотреть активные приглашения, принять или отклонить приглашение, а также просмотреть своё расписание встреч.

Документация пользователя была подготовлена в формате README отдельного раздела пользовательской документации. Она включает следующие разделы:

1. Общее описание приложения.
2. Регистрация и авторизация.
3. Настройка личного профиля.
4. Создание встречи.
5. Просмотр приглашений.
6. Ответ на приглашение.
7. Просмотр расписания.
8. Часто задаваемые вопросы.

Общее описание приложения

Приложение предназначено для удобного планирования встреч между сотрудниками компании. Система позволяет создавать встречи в фиксированных часовых слотах, приглашать участников и отслеживать их ответы.

Регистрация и авторизация

Для начала работы пользователь должен создать аккаунт. При регистрации указываются имя, email и пароль. После успешной регистрации пользователь может выполнить вход в систему. При авторизации сервер проверяет введённые данные и возвращает токен доступа, который используется для работы с защищёнными разделами приложения.

Настройка личного профиля

После входа пользователь может открыть раздел профиля. В профиле отображаются данные сотрудника: имя, email и аватар. Пользователь может редактировать доступные данные профиля, после чего изменения сохраняются на сервере.

Создание встречи

Для создания встречи пользователь открывает экран создания встречи, указывает тему, описание, дату, время, место проведения и выбирает участников из списка сотрудников. Важным условием является то, что встреча создаётся только на целый часовой слот, например 9:00–10:00 или 14:00–15:00.

После заполнения формы данные отправляются на сервер. Сервер проверяет корректность данных и наличие конфликтов по времени. Если ошибок нет, встреча сохраняется, а выбранным сотрудникам создаются приглашения.

Просмотр приглашений

В разделе приглашений пользователь видит список активных приглашений на встречи. Каждая карточка приглашения содержит название встречи, время, место проведения и организатора. Пользователь может открыть приглашение и принять или отклонить его.

Ответ на приглашение

Для ответа на приглашение пользователь нажимает кнопку «Принять» или «Отклонить». После выбора статус приглашения сохраняется на сервере. Если пользователь принимает приглашение, встреча отображается в его расписании.

Просмотр расписания

В разделе расписания пользователь может просматривать свои встречи. Расписание может отображаться в разрезе дня, недели или месяца. Это позволяет сотруднику планировать рабочее время и видеть занятые временные слоты.

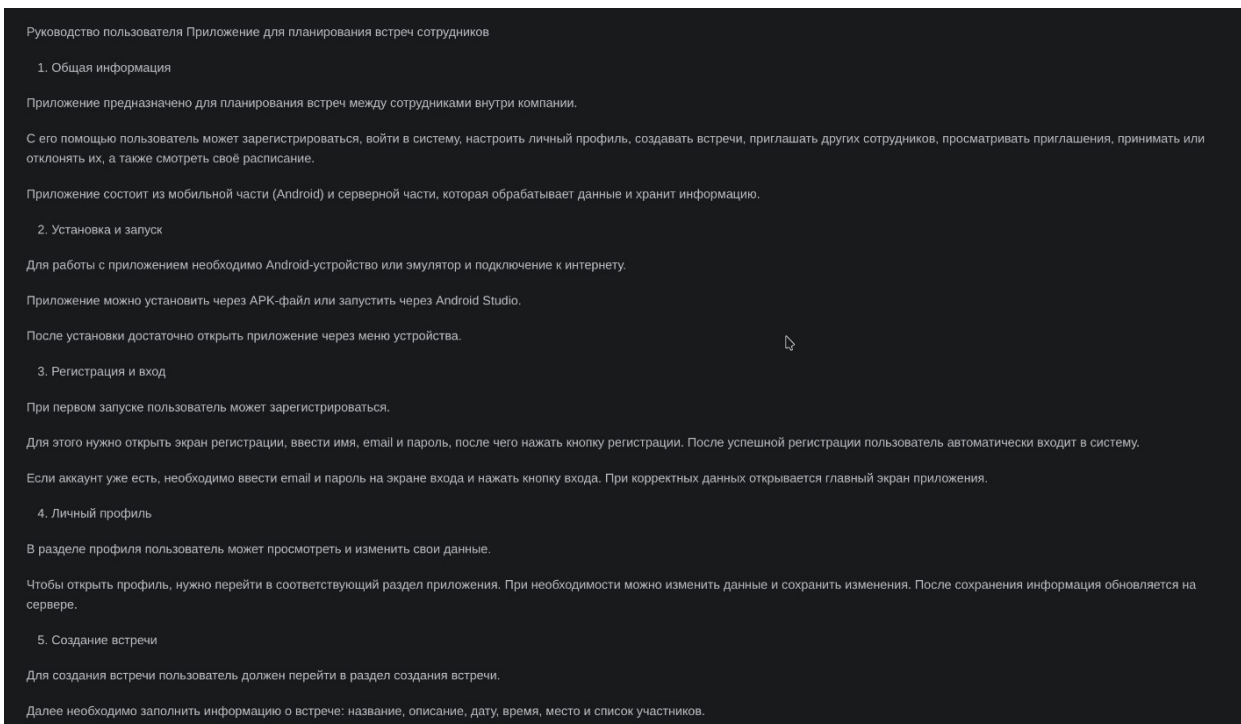


Рис.7 — user_guide.md

6 ЗАКЛЮЧЕНИЕ

В ходе выполнения практических работ №9–10 был рассмотрен процесс организации разработки программного продукта на примере приложения для планирования встреч сотрудников внутри компании.

В рамках работы была выбрана и обоснована методология разработки. Оптимальным решением стал комбинированный подход, включающий Agile и итерационную модель. Такой подход подходит для данного проекта, так как приложение состоит из нескольких функциональных модулей, которые удобно реализовывать и проверять поэтапно.

Также были созданы и использованы удалённые Git-репозитории на платформе GitHub. Проект был разделён на две части: серверную часть и клиентское Android-приложение. Такое разделение позволяет независимо разрабатывать backend и frontend, а затем интегрировать их через REST API.

В ходе работы был подобран набор инструментов разработки. Для клиентской части использовались Kotlin, Android Studio, Jetpack Compose, Retrofit, Hilt и DataStore. Для серверной части использовались Java, Spring Boot, Spring Data JPA, Spring Security, JWT, Liquibase, Maven, PostgreSQL и Docker.

Кроме того, была подготовлена документация разработчика и пользователя. Документация разработчика описывает архитектуру проекта, структуру клиентской и серверной частей, модели данных, API и базу данных. Документация пользователя объясняет основные действия в приложении: регистрацию, авторизацию, настройку профиля, создание встречи, просмотр приглашений, ответ на приглашения и просмотр расписания.

Полученные результаты позволяют обеспечить дальнейшее развитие, сопровождение и использование программного продукта.